Tab 1

🔹 Experiment 1: Study of Web Analytics Tools

✅ Aim

To study open-source web analytics tools like **Matomo**, **Open Web Analytics**, **AWStats**, **Countly**, and **Plausible**.

📘 Short Theory (Exam Ready)

Web analytics is the process of **collecting, analyzing, and reporting website data** to understand user behavior.

👉 It helps in:

- Tracking visitors
- Measuring performance
- Improving user experience
- Making business decisions

🔧 Tools Overview (VERY IMPORTANT for viva)

1. Matomo

- Open-source alternative to Google Analytics
- Self-hosted (data privacy)
- Tracks visits, clicks, conversions

👉 **Key point:** Full control over data

---

2. Open Web Analytics (OWA)

- Tracks users, pages, clicks
- Similar to Google Analytics
- Supports heatmaps

👉 **Key point:** Simple and customizable

---

3. AWStats

- Works using **server log files**
- No need to insert tracking code
- Shows traffic, OS, browsers

👉 **Key point:** Log-based analytics

---

4. Countly

- Real-time analytics
- Used in mobile + web apps
- Tracks sessions, crashes

👉 **Key point:** Product analytics focus

---

5. Plausible

- Lightweight & privacy-friendly
- No cookies
- Simple dashboard

👉 **Key point:** GDPR-friendly analytics

⚖️ Quick Comparison (important)

Tool	Type	Special Feature
Matomo	Self-hosted	Privacy control
OWA	Open-source	Heatmaps
AWStats	Log-based	No JS needed
Countly	Real-time	Mobile analytics
Plausible	Lightweight	No cookies

🧠 Viva Questions (must prepare)

1. What is web analytics?

➡️ Process of collecting and analyzing website data.

2. Why use open-source tools?

➡️ Free, customizable, better privacy.

3. Difference: AWStats vs Matomo

➡️

- AWStats → log-based
- Matomo → JavaScript tracking

4. Which tool is privacy-focused?

➡️ Plausible / Matomo

5. What is bounce rate?

➡️ % of users leaving after one page.

Tab 2

🔹 Experiment 2: Study of Semantic Web Tools

✅ Aim

To study semantic web tools like **Apache TinkerPop**, **RDFLib**, **Apache Jena**, and **Protégé**.

📘 Short Theory (Exam Ready)

The **Semantic Web** is an extension of the web where data is **structured and machine-readable**.

👉 Instead of just displaying data, it helps machines **understand relationships** between data.

🔑 Key concept:

- Uses **RDF (Resource Description Framework)**
- Data is stored in **triples** → **(Subject, Predicate, Object)**
  👉 Example:
  `Mrudul → studies → IT`

🔧 Tools Overview (Viva Focus)

1. Apache Jena

- Java-based framework
- Used to create and query RDF data
- Supports SPARQL queries

👉 **Key point:** RDF + SPARQL processing

---

2. RDFLib

- Python library
- Used to work with RDF data
- Easy for small projects

👉 **Key point:** Python-based RDF handling

---

3. Protégé

- GUI tool (no coding needed)
- Used to create **ontologies** (data models)

👉 **Key point:** Visual ontology design

---

4. Apache TinkerPop

- Graph-based framework
- Uses Gremlin query language

👉 **Key point:** Graph traversal

🧠 Important Terms (must know)

- **RDF** → Data format using triples
- **Ontology** → Structure of knowledge (like schema)
- **SPARQL** → Query language for RDF
- **Triple** → Subject + Predicate + Object

💻 Simple Example (RDF in Python using RDFLib)

```
from rdflib import Graph

g = Graph()

g.add(("Mrudul", "studies", "IT"))

for s, p, o in g:
  print(s, p, o)
```

👉 Output:

```
Mrudul studies IT
```

⚖️ Quick Comparison

Tool	Language	Use
Jena	Java	RDF + SPARQL
RDFLib	Python	RDF handling
Protégé	GUI	Ontology design
TinkerPop	Java	Graph processing

🧠 Viva Questions

1. What is Semantic Web?

➡️ Web where data is machine-understandable.

2. What is RDF?

➡️ Data model using triples.

3. What is SPARQL?

➡️ Query language for RDF data.

4. What is ontology?

➡️ Structure that defines relationships between data.

5. Difference: RDF vs XML

➡️

- RDF → Meaning (semantic)
- XML → Structure only

Tab 3

🔹 Experiment 3: Hello World using TypeScript

✅ Aim

To write and execute a simple **Hello World program using TypeScript**.

📘 Theory (Short & Crisp)

TypeScript is a **superset of JavaScript** that adds:

- Static typing
- Better error checking
- Improved code readability

👉 It is developed by Microsoft

👉 TypeScript code is converted into JavaScript using a compiler called **tsc**

⚙️ Steps to Run (Practical)

1. Install TypeScript:

npm install -g typescript

2. Create file:

hello.ts

3. Compile:

tsc hello.ts

4. Run:

node hello.js

💻 Code

```
let message: string = "Hello World";
console.log(message);
```

▶️ Output

Hello World

🧠 Viva Questions

1. What is TypeScript?

➡️ Superset of JavaScript with static typing.

2. Why use TypeScript?

➡️ Better error detection and code quality.

3. What is `tsc`?

➡️ TypeScript compiler.

4. Does browser run TypeScript directly?

➡️ ❌ No, it runs JavaScript.

5. Difference: JS vs TypeScript

➡️

- JS → dynamically typed
- TS → statically typed

Tab 4

🔹 Experiment 4: Calculator using TypeScript

✅ Aim

To implement a simple calculator using **TypeScript** for performing basic arithmetic operations.

📘 Theory (Short & Useful)

A calculator program demonstrates:

- Functions
- Input handling
- Arithmetic operations

👉 TypeScript improves this by adding **type safety** (e.g., numbers only).

💻 Code (Simple Version)

```
function add(a: number, b: number): number {
  return a + b;
}

function subtract(a: number, b: number): number {
  return a - b;
}

function multiply(a: number, b: number): number {
  return a * b;
}

function divide(a: number, b: number): number {
  if (b === 0) {
    console.log("Cannot divide by zero");
    return 0;
  }
  return a / b;
}
```

```
// Example usage
let x: number = 10;
let y: number = 5;

console.log("Addition:", add(x, y));
console.log("Subtraction:", subtract(x, y));
console.log("Multiplication:", multiply(x, y));
console.log("Division:", divide(x, y));
```

▶️ Output

Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2

⚙️ Steps to Run

Same as previous:

tsc calculator.ts
node calculator.js

🧠 Viva Questions

1. Why use functions here?

➡️ To make code modular and reusable.

2. What is return type `: number`?

➡️ It ensures function returns a number.

3. Why check division by zero?

➡️ To avoid runtime error.

4. Can we take user input?

➡️ Yes, using prompt or readline.

5. Advantage over JavaScript?

➡️ Type safety (prevents wrong inputs).

Tab 5

🔹 Experiment 5: Inheritance using TypeScript

✅ Aim

To implement **inheritance** using **TypeScript**.

📘 Theory (Short & Strong)

Inheritance is an OOP concept where:
👉 One class (**child**) acquires properties of another class (**parent**)

🔑 Benefits:

- Code reuse
- Better structure
- Supports OOP concepts

👉 Keyword used: `extends`

💻 Code Example

```
class Person {
  name: string;

  constructor(name: string) {
     this.name = name;
  }

  display(): void {
     console.log("Name:", this.name);
  }
}

class Student extends Person {
  rollNo: number;

  constructor(name: string, rollNo: number) {
     super(name);  // call parent constructor
     this.rollNo = rollNo;
  }
```

```
  show(): void {
    console.log("Roll No:", this.rollNo);
  }
}

// Object creation
let s = new Student("Mrudul", 40);

s.display();  // from parent
s.show();     // from child
```

▶️ Output

Name: Mrudul
Roll No: 40

⚙️ Key Points (Exam Focus)

- `extends` → used for inheritance
- `super()` → calls parent constructor
- Child class can access parent methods

🧠 Viva Questions

1. What is inheritance?

➡️ Mechanism where one class acquires properties of another.

2. What is `extends`?

➡️ Keyword to inherit a class.

3. Why use `super()`?

➡️ To call parent constructor.

4. Can child access parent methods?

➡️ Yes.

5. Types of inheritance?

➡️ Single, Multiple (not directly in TS), Multilevel

Tab 6

Experiment 6: Access Modifiers in TypeScript

✅ Aim

To implement **access modifiers** using **TypeScript**.

📘 Theory (Short & Important)

Access modifiers control **visibility of variables and methods** in a class.

🔑 Types:

1. **public**
2. **private**
3. **protected**

⚙️ Explanation

1. Public

- Accessible **everywhere**
- Default modifier

2. Private

- Accessible **only inside the same class**

3. Protected

- Accessible **within class + child class**

💻 Code Example

```
class Person {
  public name: string;
  private age: number;
  protected city: string;
```

```
  constructor(name: string, age: number, city: string) {
      this.name = name;
      this.age = age;
      this.city = city;
  }

  showDetails(): void {
      console.log("Name:", this.name);
      console.log("Age:", this.age);
      console.log("City:", this.city);
  }
}

class Student extends Person {
  display(): void {
      console.log("Accessing protected city:", this.city);
      // console.log(this.age); ❌ not allowed (private)
  }
}
let s = new Student("Mrudul", 20, "Mumbai");

console.log(s.name);   // ✅ public
// console.log(s.age); ❌ private
// console.log(s.city); ❌ protected

s.display();
s.showDetails();
```

▶️ Output

Mrudul
Accessing protected city: Mumbai
Name: Mrudul
Age: 20
City: Mumbai

⚖️ Quick Comparison

Modifier	Same Class	Child Class	Outside Class
public	✔	✔	✔
private	✔	✖	✖
protected	✔	✔	✖

🧠 Viva Questions

1. What are access modifiers?

➡️ They control visibility of class members.

2. Default modifier?

➡️ public

3. Difference: private vs protected

➡️

- private → only same class
- protected → class + child class

4. Can private be accessed outside?

➡️ ❌ No

5. Why use access modifiers?

➡️ Data hiding and security

Tab 7

🔹 Experiment 7: Hello World using AngularJS

✅ Aim

To create a simple Hello World application using **AngularJS** and demonstrate:

- `ng-controller`
- `ng-model`
- Expressions

📘 Theory (Short & Strong)

AngularJS is a JavaScript framework used to build **dynamic web applications**.

👉 It follows:

- **MVC architecture**
- Data binding between model and view

🔑 Key Concepts

1. `ng-app`

➡️ Initializes AngularJS application

2. `ng-controller`

➡️ Connects HTML with JavaScript controller

3. `ng-model`

➡️ Binds input field with variable

4. Expressions `{{ }}`

➡️ Display dynamic data

💻 Code

```
<!DOCTYPE html>
<html ng-app="">
<head>
  <title>AngularJS Hello</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body>

  <h2>AngularJS Hello World</h2>

  <div ng-controller="myController">
     Enter your name:
     <input type="text" ng-model="name">

     <h3>Hello {{name}}</h3>
  </div>

  <script>
     function myController($scope) {
        $scope.name = "Mrudul";
     }
  </script>

</body>
</html>
```

▶️ Output

- Initially → **Hello Mrudul**
- When user types → updates automatically

👉 This is called **two-way data binding**

⚙️ Important Points (Exam)

- No page reload required
- Real-time update
- Data binding between UI & model

🧠 Viva Questions

1. What is AngularJS?

➡️ JavaScript framework for dynamic web apps.

2. What is `ng-model`?

➡️ Binds input to variable.

3. What are expressions?

➡️ `{{ }}` used to display data.

4. What is two-way data binding?

➡️ Changes in UI ↔ automatically reflect in model.

5. What is controller?

➡️ JavaScript function that manages data.

Tab 8

🔹 Experiment 8: Event Handling using AngularJS

✅ Aim

To implement **event handling** using **AngularJS**.

📘 Theory (Short & Important)

Event handling means responding to user actions like:

- Click
- Key press
- Mouse events

👉 AngularJS provides **directives** for handling events.

🔑 Common Event Directives

- `ng-click` → button click
- `ng-change` → input change
- `ng-mouseover` → mouse hover

💻 Code Example (ng-click)

```
<!DOCTYPE html>
<html ng-app="">
<head>
  <title>Event Handling</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-controller="myController">

  <h2>AngularJS Event Handling</h2>
```

```
  <button ng-click="count = count + 1">Click Me</button>

  <h3>Button Clicked {{count}} times</h3>

  <script>
     function myController($scope) {
        $scope.count = 0;
     }
  </script>

</body>
</html>
```

▶️ Output

- Initially → `0 times`
- Each click → count increases

⚙️ Key Concept

👉 `ng-click` directly binds button click to logic
👉 No need for separate JS event listeners

🧠 Viva Questions

1. What is event handling?
➡️ Responding to user actions.

2. What is `ng-click`?
➡️ Executes code on button click.

3. Difference: JS vs AngularJS event handling

- JS → addEventListener
- AngularJS → directives (ng-click)

4. Can we handle multiple events?
➡️ Yes.

5. Advantage?
➡️ Simpler and cleaner code

Tab 9

Experiment 9: Student Record Application using AngularJS

✅ Aim

To create a simple **Student Record application** using **AngularJS**.

📘 Theory (Short & Practical)

This application demonstrates:

- **Data binding**
- **Arrays (list of students)**
- **Displaying dynamic data in table**

👉 AngularJS automatically updates UI when data changes.

🔑 Concepts Used

- `ng-app` → start app
- `ng-controller` → control logic
- `ng-repeat` → loop through data
- Expressions `{{ }}` → display values

💻 Code

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head>
  <title>Student Record</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-controller="studentController">

  <h2>Student Records</h2>
```

```
  <table border="1">
    <tr>
      <th>Name</th>
      <th>Roll No</th>
      <th>Branch</th>
    </tr>

    <tr ng-repeat="s in students">
      <td>{{s.name}}</td>
      <td>{{s.roll}}</td>
      <td>{{s.branch}}</td>
    </tr>
  </table>

  <script>
    var app = angular.module("studentApp", []);

    app.controller("studentController", function($scope) {
      $scope.students = [
        {name: "Mrudul", roll: 101, branch: "IT"},
        {name: "Amit", roll: 102, branch: "CS"},
        {name: "Neha", roll: 103, branch: "ENTC"}
      ];
    });
  </script>

</body>
</html>
```

▶️ Output

A table displaying:

- Name
- Roll No
- Branch

⚙️ Key Points (Exam Focus)

- `ng-repeat` → loops through array
- Data automatically reflects in UI
- No manual DOM manipulation

🧠 Viva Questions

1. What is `ng-repeat`?

➡️ Used to loop through list/array.

2. What is `$scope`?

➡️ Object that connects controller and view.

3. How is data stored?

➡️ In array of objects.

4. Advantage of AngularJS here?

➡️ Automatic UI update.

5. Can we add new student dynamically?

➡️ Yes.

Tab 10

Experiment 10: AJAX using JavaScript

✅ Aim

To implement **AJAX** using JavaScript to fetch data from server without reloading the page.

📘 Theory (Short & Strong)

AJAX = Asynchronous JavaScript and XML

👉 It allows:

- Updating web page **without reload**
- Fetching data **in background**

🔑 Key Concept

👉 Uses:

- `XMLHttpRequest` OR modern `fetch()`

👉 Flow:

1. Request sent to server
2. Server sends response
3. Page updates dynamically

💻 Code (Using XMLHttpRequest)

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX Example</title>
</head>
<body>
  <h2>AJAX Demo</h2>
  <button onclick="loadData()">Load Data</button>

  <p id="result"></p>

  <script>
    function loadData() {
      var xhttp = new XMLHttpRequest();
```

```
        xhttp.onreadystatechange = function() {
            if (this.readyState == 4 && this.status == 200) {
                document.getElementById("result").innerHTML = this.responseText;
            }
        };

        xhttp.open("GET", "data.txt", true);
        xhttp.send();
    }

  </script>
</body>
</html>
```

▶️ Output

- Clicking button loads content from `data.txt`
- No page reload

⚙️ Important Points

- `readyState == 4` → request complete
- `status == 200` → success
- `true` → asynchronous

🧠 Viva Questions

1. What is AJAX?

➡️ Technique to update web page without reload.

2. What is asynchronous?

➡️ Works in background without blocking.

3. What is XMLHttpRequest?

➡️ Object used to send HTTP requests.

4. Alternative to XMLHttpRequest?

➡️ fetch() API

5. Advantage of AJAX?

➡️ Faster and better user experience

Tab 11

Experiment 11: AJAX for User Validation

✅ Aim

To implement **user validation using AJAX** and display result on the same page without reload.

📘 Theory (Short & Important)

👉 This experiment uses:

- AJAX to send user input to server
- Server checks validity
- Result shown **on same page**

👉 Example:

- Username exists → show message
- Username not exists → show different message

🔑 Concept Used

- `XMLHttpRequest`
- DOM manipulation
- Dynamic response

💻 Code (Frontend)

```
<!DOCTYPE html>
<html>
<head>
  <title>User Validation</title>
</head>

<body>

  <h2>User Validation using AJAX</h2>
```

```
  Enter Username:
  <input type="text" id="username" onkeyup="validateUser()">

  <p id="result"></p>

  <script>
    function validateUser() {
      var user = document.getElementById("username").value;

      var xhttp = new XMLHttpRequest();

      xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          document.getElementById("result").innerHTML = this.responseText;
        }
      };

      xhttp.open("GET", "checkUser.php?name=" + user, true);
      xhttp.send();
    }
  </script>

</body>
</html>
```

💻 Server Code (PHP Example)

```
<?php
$name = $_GET['name'];

$users = ["Mrudul", "Amit", "Neha"];

if(in_array($name, $users)) {
  echo "Username already exists";
} else {
  echo "Username available";
}
?>
```

▶️ Output

- Typing name:
  - Existing → "Username already exists"
  - New → "Username available"
- No page reload

⚙️ Key Points (Exam)

- `onkeyup` → triggers validation
- AJAX sends request dynamically
- Result shown instantly

🧠 Viva Questions

1. What is user validation?

➡️ Checking whether input data is correct or valid.

2. Why AJAX used here?

➡️ To validate without page reload.

3. What is `onkeyup`?

➡️ Event triggered when user types.

4. What method is used?

➡️ GET method.

5. Advantage?

➡️ Real-time validation

Tab 12

Experiment 12: REST API using MongoDB

✅ Aim

To build a **RESTful API** using **Node.js**, **Express.js**, and **MongoDB**.

📘 Theory (Short & Strong)

🔹 REST API

REST (Representational State Transfer) is used to:

- Perform CRUD operations
- Communicate between client & server

🔹 CRUD Operations

- **Create → POST**
- **Read → GET**
- **Update → PUT**
- **Delete → DELETE**

⚙️ Requirements

npm init -y
npm install express mongoose body-parser

💻 Code

```
const express = require("express");
const mongoose = require("mongoose");
const bodyParser = require("body-parser");

const app = express();
app.use(bodyParser.json());

// Connect MongoDB
```

```
mongoose.connect("mongodb://localhost:27017/school");

// Schema
const studentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  branch: String
});

const Student = mongoose.model("Student", studentSchema);

// GET
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// POST
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  await student.save();
  res.status(201).json(student);
});

// PUT
app.put("/students/:id", async (req, res) => {
  const updated = await Student.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );
  res.json(updated);
});

// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({ message: "Deleted successfully" });
});

// Server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

▶️ Output

Using Postman / browser:

- GET → fetch students
- POST → add student
- PUT → update student
- DELETE → remove student

⚙️ Key Points (Exam Focus)

- Express handles routes
- MongoDB stores data
- Mongoose → connects Node & MongoDB

🧠 Viva Questions

1. What is REST API?

➡️ API that follows HTTP methods for CRUD operations.

2. What is Mongoose?

➡️ ODM to interact with MongoDB.

3. Difference: SQL vs MongoDB

➡️

- SQL → tables
- MongoDB → documents

4. What is JSON?

➡️ Data format used in APIs.

5. Why use Express?

➡️ Simplifies server creation.

Tab 13

Experiment 13: Feedback Form using Flask

✅ Aim

To design a **feedback form** using **Flask**.

📘 Theory (Short & Important)

Flask is a lightweight Python web framework used to:

- Build web applications
- Handle forms
- Process user input

👉 Works using:

- Routes
- Templates (HTML)

⚙️ Requirements

pip install flask

💻 Code

🔹 `app.py`

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('form.html')

@app.route('/submit', methods=['POST'])
def submit():
```

```
    name = request.form['name']
    feedback = request.form['feedback']
    return f"Thank you {name}, your feedback is received!"

if __name__ == '__main__':
    app.run(debug=True)
```

🔹 `templates/form.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
</head>

<body>
    <h2>Feedback Form</h2>

    <form action="/submit" method="POST">
        Name: <input type="text" name="name"><br><br>
        Feedback: <textarea name="feedback"></textarea><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

▶️ Output

- Form displayed
- On submit → shows:

Thank you Mrudul, your feedback is received!

⚙️ Key Points

- `@app.route()` → defines URL
- `POST` → used to send form data
- `request.form` → access user input

🧠 Viva Questions

1. What is Flask?

➡️ Lightweight Python web framework.

2. What is route?

➡️ URL mapping to function.

3. Why POST method?

➡️ To send form data securely.

4. What is template?

➡️ HTML file used in Flask.

5. How data is accessed?

➡️ Using `request.form`

Tab 14

Experiment 14: Quiz Result App using Flask

✅ Aim

To design a simple **Quiz Result App** using **Flask**.

📘 Theory

A Quiz Result App takes input such as marks or score from the user, processes it on the server, and displays the result.

This experiment demonstrates:

- Flask routing
- HTML forms
- POST method
- Input processing
- Dynamic result display

⚙️ Requirement

pip install flask

💻 Code

`app.py`

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('quiz.html')

@app.route('/result', methods=['POST'])
def result():
```

```
    marks = int(request.form['marks'])

    if marks >= 75:
        grade = "Distinction"
    elif marks >= 60:
        grade = "First Class"
    elif marks >= 40:
        grade = "Pass"
    else:
        grade = "Fail"

    return f"Your Marks: {marks} <br> Result: {grade}"

if __name__ == '__main__':
    app.run(debug=True)
```

`templates/quiz.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Quiz Result App</title>
</head>
<body>
    <h2>Quiz Result App</h2>

    <form action="/result" method="POST">
        Enter Marks:
        <input type="number" name="marks" required>
        <br><br>
        <input type="submit" value="Show Result">
    </form>
</body>
</html>
```

▶️ Output

If user enters:

- `85` → Distinction
- `65` → First Class
- `45` → Pass
- `30` → Fail

🧠 Viva Questions

1. Why is Flask used here?

➡️ Flask is used to build a simple web app and process form input.

2. Why POST method is used?

➡️ To send form data from HTML to Flask server.

3. What is `request.form`?

➡️ It is used to get input values submitted from the form.

4. What is route in Flask?

➡️ Route maps a URL to a Python function.

5. Why is `templates` folder used?

➡️ To store HTML pages used by Flask.

Short Weather App version

If examiner specifically asks for **Weather App**, you can say this version:

`app.py`

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('weather.html')

@app.route('/weather', methods=['POST'])
def weather():
```

```
    city = request.form['city']
    return f"Weather report for {city}: Sunny, 30°C"

if __name__ == '__main__':
    app.run(debug=True)
```

templates/weather.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Weather App</title>
</head>
<body>
    <h2>Weather App</h2>
    <form action="/weather" method="POST">
        Enter City:
        <input type="text" name="city" required>
        <input type="submit" value="Check Weather">
    </form>
</body>
</html>
```